CFDGraph: Privacy-Preserving Graph Processing for Large-Scale Collaborative Fraud Detection

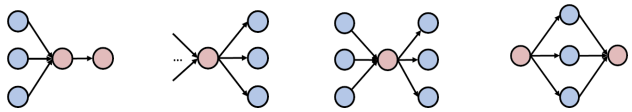
Qiulin Wu[†*], Amelie Chi Zhou[*], Tristan Allard[‡], Shadi Ibrahim[§], Yuhong Feng[†], Lichun Li[¶] and Amr El Abbadi[‖]
[†]College of CSSE, Shenzhen University [*]Hong Kong Baptist University [‡]Univ. Rennes, CNRS, IRISA
[§]Inria, Univ. Rennes, CNRS, IRISA [¶]Ant Digital Technologies, Ant Group [‖]UC Santa Barbara

***Abstract*—Fraud detection is a critical task in finance and e-commerce, but fraudsters increasingly evade detection by distributing their activities across multiple institutions. While this makes collaborative fraud detection (CFD) essential, it is hindered by two fundamental barriers: stringent data privacy regulations and constrained communication networks. Existing systems are impractical, as they either incur prohibitive overheads or cannot attain the high detection accuracy required to manage financial risk. This paper introduces *CFDGraph*, the first privacy-preserving graph processing system designed to overcome these challenges by synergistically balancing accuracy, privacy, and communication efficiency. CFDGraph's novelty lies in three interconnected innovations. First, it employs differential privacy with adaptive quantization to simultaneously reduce both privacy-induced noise and communication overhead. Second, it introduces accuracy-aware noise mitigation techniques to preserve the high recall essential for fraud detection. Third, it implements privacy-aware message compression to dramatically reduce data transmission volume. Evaluations show CFDGraph achieves 100% recall in identifying top-k fraudulent accounts while reducing manual investigation costs by up to 99.5% compared to state-of-the-art approaches.**

I. Introduction

Existing e-commerce platforms and financial institutions strive to detect fraud entities to mitigate risk and minimize financial loss [1], [2], [3]. Large e-commerce platforms, such as Taobao [3], serve hundreds of millions of active users daily, leading to a substantial volume of transactions. According to the latest statistics, Chinese banks processed 279 billion electronic payment transactions in 2022 alone, with a transaction amount of 311 trillion yuan [4]. The sheer scale of these transactions complicates the task of detecting fraud for both high accuracy and high recall.

To manage this complexity, industrial practices have adopted a two-step approach [1]. The first step utilizes random walk algorithms like PageRank [5], [6] to calculate a "fraud score" for each account. In the second step, accounts with high scores undergo deeper analysis through expert investigation or advanced machine learning techniques like Graph Neural Networks (GNNs) [7], [8], [9]. This approach greatly reduces the number of accounts requiring costly, in-depth analysis. While considerable effort has been directed toward enhancing the second step with various GNN models [3], [7], [8], [9], the effectiveness of the entire process is fundamentally limited by the quality of the first step. *Any potential fraudster missed during the initial risk-scoring phase is never investigated, making high recall in the first step the paramount objective.* This paper concentrates on enhancing this crucial first step, aiming to improve the accuracy and recall of finding the top-k riskiest accounts.



(a) Fan-in (b) Fan-out (c) Gather-Scatter (d) Scatter-Gather

Fig. 1: Typical transaction patterns in money laundering. Red nodes are suspicious accounts [10].

Identifying the top-k risky accounts is effectively a graph computing problem, where accounts are nodes and transactions are edges. Random walk algorithms excel at detecting common fraudulent patterns [5], [11], such as the "Fan-in" scheme where multiple accounts transfer funds to a single destination—a strong indicator of money laundering (e.g., Figure 1). Algorithms like Personalized PageRank (PPR) [12] can detect such patterns by propagating risk scores to identify high-traffic recipients. However, modern fraudsters now evade detection by exploiting *institutional silos*, distributing their transactions across multiple organizations (e.g., Figure 2). An account that appears legitimate within a single institution may reveal significant risk when its cross-institutional activities are aggregated. This evolution makes *Collaborative Fraud Detection (CFD)* imperative. However, effective CFD is currently impractical due to two fundamental and interrelated barriers.

First, Data Privacy Challenges: Institutions cannot directly share transaction data due to strict data sovereignty policies and regulatory mandates like GDPR [13]. While privacy-preserving technologies exist, they present an intractable trade-off. Fully Homomorphic Encryption (FHE) [14] enables computation on encrypted data but incurs prohibitive computational and bandwidth costs for large graphs, making real-time fraud detection infeasible. Meanwhile, naive applications of Differential Privacy (DP) introduce so much statistical noise that they unacceptably degrade detection accuracy [13], causing substantial financial losses.

Second, Data Communication Challenges: Financial institutions are interconnected via secure but resource-constrained channels like VPNs [15], where bandwidth is limited (e.g., around 10MB/s [13]). On the other hand, real-world transaction graphs can contain $> 10^9$ edges [16], making the

limited bandwidth a major performance bottleneck. Standard distributed graph processing optimizations, such as graph partitioning [17], [18], [19], [20] and message filtering [21], are incompatible with CFD's unique constraints. For instance, data ownership policies prevent the re-partitioning of institutional subgraphs to minimize cross-institutional data communications, and privacy-aware message filtering requires the inspection of sensitive data, creating privacy violations.

To solve this, we introduce *CFDGraph*, a privacy-preserving graph system designed explicitly for CFD problems. CFDGraph integrates three key innovations: ① **Differential Privacy (DP) with Adaptive Quantization:** CFDGraph adopts DP to protect inter-institution data exchanges with minimal overhead. We introduce an *adaptive, non-uniform quantization* scheme that dynamically learns the data distributions in each iteration. This novel application simultaneously reduces the data range to lower DP sensitivity (hence lower noise) and shrinks message sizes to cut communication overhead. ② **Accuracy-Aware Noise Mitigation:** To further counteract DP-induced accuracy loss, we develop two techniques. Critically, we introduce *range-partitioned combiners*, a novel heuristic that groups messages by value and allocates the privacy budget non-uniformly to protect the most critical information, alongside standard data *clipping*. Clipping limits the maximum influence of any single vertex, further lowering global sensitivity and noise. ③ **Privacy-Aware Communication Compression:** We design data-oblivious compression techniques, including *vertex reindexing* and *message aggregation*, that drastically reduce data transmission size without privacy leaks.

We implement CFDGraph within the widely-used Pregel [22] graph processing system and evaluate its effectiveness on real-world social and financial graphs against state-of-the-art (SOTA) privacy-preserving techniques [13], [21]. Our evaluations demonstrate CFDGraph's practical viability and significant advantages. For instance, on an Anti-Money Laundering (AML) graph, CFDGraph achieves **100% recall** of the top-0.1% risky accounts by retrieving only 0.36% of total accounts, leading to a **99.5% reduction** in manual investigation costs compared to the 73.3% account retrieval required by SOTA systems. Furthermore, it reduces cross-institution data transfers by 75%-98% on large graphs and maintains high precision (78.1%-100%) even under stringent DP guarantees (ϵ=1.0), confirming its suitability for real-world CFD applications.

In summary, this paper makes the following contributions:

- The first to model collaborative fraud detection as privacy and communication-constrained graph optimization, explicitly addressing how fraudsters exploit institutional silos.
- Proposes and implements CFDGraph, the first *practical* and *deployable* solution for CFD problems that simultaneously provides differential privacy guarantees, low latency, and production-grade accuracy.
- Provides formal proof for the DP guarantees of CFDGraph and theoretically analyzes the utility impact of each optimization technique of CFDGraph.

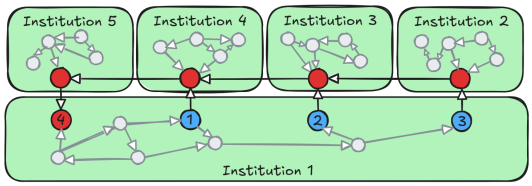


Fig. 2: An example of cross-institutional transactions

- By demonstrating real-world viability with 100% recall and 99.5% cost reduction in fraud investigations, CFDGraph establishes a new benchmark for practical privacy-preserving fraud detection and makes real-world impact.

II. Background and Preliminaries

A. Collaborative Fraud Detection

Fraud detection algorithms typically employ a two-step detection approach [1] to reduce complexity: 1) calculating a graph-based "risk score" for all accounts, and 2) in-depth investigation of high-scoring targets. This paper focuses on the first stage, which utilizes centrality measures to identify topological patterns indicative of money laundering. As illustrated in Figure 1, these patterns include *Fan-in*, where multiple accounts transfer funds to a single destination, and *Fan-out*, where one account distributes funds to many others. These patterns may overlap, forming more complex schemes like *Scatter-Gather* and *Gather-Scatter*.

While algorithms like PageRank effectively detect these structures within a single dataset, modern fraudsters evade detection by fragmenting these patterns across institutional silos. Figure 2 illustrates a complex cross-institutional fan-in scheme. Viewed in isolation, Institution 1 observes disparate accounts (1, 2, 3) transferring funds to distinct external institutions (Inst. 2, 3, 4), while account 4 receives a seemingly unrelated transaction from Inst. 5. Locally, these events appear disjoint and legitimate. However, a collaborative view reveals a hidden **transaction chain** flowing through Institutions 2, 3, 4, and 5 that connects the sources back to the destination. This exposes a sophisticated laundering cycle that can only be identified via *Collaborative Fraud Detection (CFD)*.

B. Graph Processing for Fraud Detection

Graphs are powerful tools for data analytics, and have been widely adopted by finance and e-commerce platforms to model transactions and detect fraudulent behaviors [23]. By representing entities like user accounts as nodes and their interactions as edges, graph-based approaches can effectively uncover hidden patterns in collaborative fraud schemes. For example, Alibaba's eRiskCom platform [1] uses the SALSA algorithm to rank users by risk scores, efficiently identifying core members of fraudulent communities for further investigation. Similarly, random walk algorithms like Label Propagation [11] and PageRank [5], [6] have proven effective in detecting risky communities. Compared to static machine learning models, a key advantage of these methods is their ability to dynamically adapt to evolving graph structures without requiring model retraining, making them well-suited for real-world fraud detection.

While existing graph-based approaches achieve good results, they overlook the critical challenges inherent in CFD: preserving data privacy when analyzing cross-institution graphs and minimizing the associated communication overhead. This work addresses these challenges by designing *privacy-preserving* and *communication-efficient* random walk algorithms tailored specifically for the demands of CFD.

III. Problem Overview

A. Data Model

Consider a large transaction network spanning across M financial institutions. The full transaction network can be considered as a single directed unweighted graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of vertices, namely user accounts, and $\mathcal{E}$ is a set of edges, namely transactions. Each institution owns a partition of the graph, denoted as $\mathcal{G}_i = (\mathcal{V}_i, \mathcal{E}_i)$, where $i \in [0, M-1]$ and $\mathcal{V}_i \subset \mathcal{V}$, $\mathcal{E}_i \subset \mathcal{E}$, $\mathcal{V}_i \cap \mathcal{V}_j = \emptyset$ and $\mathcal{E}_i \cap \mathcal{E}_j = \emptyset$ if $i \neq j$. Inter-institution edges connect two vertices belonging to different partitions and are stored in the partition of the source vertex (i.e., $(u, v) \in \mathcal{E}_i$ if $u \in \mathcal{V}_i$). To capture real-time fraud patterns, $\mathcal{G}$ is dynamically constructed from the transactions within *discrete, non-overlapping sliding windows*. Each transaction acts as an **atomic** update confined to a single window, ensuring no cross-boundary dependencies. To prevent the unbounded accumulation of sensitivity over infinite streams, each window is processed as an independent privacy unit. This design ensures that representation drift does not accumulate, as state is not persisted across boundaries. To accommodate different fraud velocities, the window duration is a configurable parameter: shorter windows (e.g., 5 minutes) detect fast-acting money laundering, while longer windows (e.g., 24 hours) capture slow-moving schemes.

B. Fraud Detection Algorithms

As mentioned in Section II, industry usually adopts a two-step fraud detection approach to reduce time complexity. This paper focuses on the risk scoring step, and adopts the popular centrality measures to compute risk scores of vertices in transaction graphs. The top-k vertices with the highest scores are identified as potential frauds, and will be investigated further in the second step.

Centrality measures are an important class of algorithms for effectively identifying suspicious transaction patterns as illustrated in Figure 1. Through iterative graph computing, these measures analyze the relationships and connectivity between nodes in a transaction graph, allowing to pinpoint suspicious behaviors. For example, the HITS algorithm [24] identifies two types of important nodes: authorities and hubs. Authorities are highly referenced nodes with a significant number of inbound links, while hubs are nodes that have numerous outbound links. Thus, authorities are good indicators of fan-in frauds (Figure 1(a)), where a node accumulates transactions from many sources, and hubs could signal fan-out frauds (Figure 1(b)), where a node disseminates transactions to many others. eRiskCom [1] and eFraudCom [2] are recent fraud detection platforms that employ similar fraud detection algorithms as ours. For example, eRiskCom employs centrality measures (e.g., PPR, SALSA) to rank high-risk users, which are effective for detecting fraudulent communities. Section VI-A introduces more details of the centrality measure algorithms adopted in this paper.

C. Privacy

Threat model. Institutions collaborate on fraud detection but remain mutually distrustful to protect their local private graphs. We consider in this work that each institution can act as an *honest-but-curious* adversary [25]: institutions follow the protocol but may exploit intermediate results or communications to infer sensitive information. The honest-but-curious attack model is common in prior and current work on privacy-preserving collaborative analytics (e.g., [26], [27], [28]) and aligns with the operational reality of cross-institutional financial collaborations where participating institutions are audited entities subject to legal and contractual constraints. Usual cryptographic techniques can be used for coping with malicious institutions (e.g., commitment schemes and zero-knowledge proofs [29]). We leave this extension to future work. Finally, we assume that an arbitrary number of adversarial institutions might collude.

Each adversarial institution has the following background knowledge. First, as commonly assumed, it knows values of all parameters except the random seeds of remote institutions. Second, it knows its local subgraph and all inter-institution edges connected to it (e.g., banks inherently know cross-institution transactions).

Edge Differential Privacy. To ensure privacy, we adopt the ϵ-edge-DP model [30], which aims to conceal the existence of individual edges (i.e., transactions) in a graph[1]. This ensures that adversaries cannot infer whether a specific transaction occurred, even when exploiting protocol outputs or shared computations (e.g., *via* membership inference attacks [31]). By design, it provides the rigorous guarantees of pure DP, protecting user privacy in distributed fraud analysis.

We consider two graphs $\mathcal{G}_1 = (\mathcal{V}_1, \mathcal{E}_1)$ and $\mathcal{G}_2 = (\mathcal{V}_2, \mathcal{E}_2)$ as *neighbors* if $\mathcal{G}_1$ differs from $\mathcal{G}_2$ by one edge, i.e., $\mathcal{V}_1 = \mathcal{V}_2$, and $|\mathcal{E}_1 - \mathcal{E}_2| = 1$ or $|\mathcal{E}_2 - \mathcal{E}_1| = 1$.

Definition 1. *[ϵ-edge-DP [30]] Let $\mathcal{G}_1$ and $\mathcal{G}_2$ be two neighboring graphs. Let* `f` *be a random function and* `Range(f)` *be the set of all possible outputs of* `f`*. If for all $\mathcal{S} \subset$* `Range(f)`*,*

$$Pr[\texttt{f}(\mathcal{G}_1) \in \mathcal{S}] \leq e^{\epsilon} Pr[\texttt{f}(\mathcal{G}_2) \in \mathcal{S}]$$

then `f` *is said to satisfy ϵ-edge-DP. ϵ is called the privacy budget. The smaller the budget, the better the privacy.*

In this paper, we consider functions as graph-based centrality measures, enforcing ϵ-edge-DP by perturbing the information flows between institutions using Laplace mechanism [32]. For a real-valued function `f`, the Laplace mechanism adds to each output a noise $Lap(\frac{\lambda}{\epsilon})$, where ϵ is the privacy budget, and λ is the L1 *global sensitivity* of `f`:

[1]Alternative differentially private models include k-edge-DP and node-DP. This work can support these more stringent models with a cost of utility.

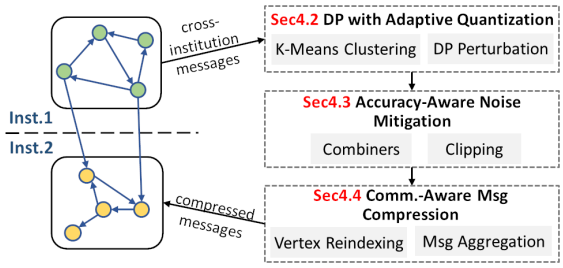


Fig. 3: Design overview of CFDGraph

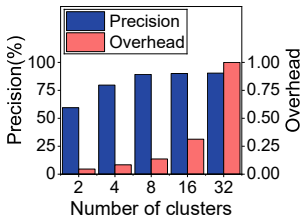


Fig. 4: Impact of #clusters on precision and overhead

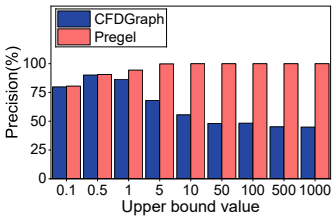


Fig. 5: Impact of Clipping on the precision of PageRank

$\lambda = \max_{\mathcal{G}_1,\mathcal{G}_2} \|\mathtt{f}(\mathcal{G}_1) - \mathtt{f}(\mathcal{G}_2)\|_1$. This sensitivity quantifies the maximum output change from adding/removing any single edge. We use $Lap(b)$ to denote $Lap(0, b)$.

IV. CFDGraph Design Details

A. CFDGraph Overview

This paper introduces CFDGraph, a distributed graph processing system designed to reconcile privacy, accuracy and efficiency for collaborative fraud detection. CFDGraph builds on top of the popular graph engine Pregel [22], which adopts the vertex-centric programming model and allows vertices to iteratively update their states through two phases: 1) *Compute Stage*: vertices update their values using messages received from in-neighbors; 2) *Communication Stage*: vertices generate new messages based on their updated states and send these messages to out-neighbors. Under this model, risk scores are computed during the *Compute Stage*. Using PageRank as an example, the compute stage is iteratively executed as follows:

$$PR_i(u) = \alpha + (1-\alpha) * \sum_{v \in N_{in}(u)} \frac{PR_{i-1}(v)}{|N_{out}(v)|} \quad (1)$$

where $PR_i(u)$ is the risk score for vertex u in iteration i, α is the constant damping factor. N_{in} and N_{out} are the sets of in/out-neighbors. $\frac{PR_{i-1}(v)}{|N_{out}(v)|}$ is included in the message from v to u. In each iteration, incoming messages to a node u are aggregated by summation. This aggregated sum is then combined with the damping factor to update the score of u. The process repeats iteratively until convergence or a predefined number of iterations is reached. Those with high scores will then be marked as potential fraudulent nodes.

Figure 3 illustrates the end-to-end workflow of CFDGraph. The system operates in three stages. First, inter-institutional messages are generated and quantized using *adaptive clustering* to minimize sensitivity (detailed in Section IV-B). Next, these messages pass through *range-partitioned combiners* and *clipping* mechanisms to mitigate noise injection (detailed in Section IV-C). Finally, the messages undergo *vertex reindexing and aggregation* to compress the payload before transmission over the constrained network (detailed in Section IV-D).

B. DP with Adaptive Quantization

1) **Design Rationale:** Various studies indicate that the degree distribution in transaction networks follows a power-law pattern [33]. As a result, the centrality-based risk scores are usually highly skewed: while scores span a wide range, most values cluster within a small one. This distribution motivates the use of quantization to remap scores into a smaller range. This technique yields a dual benefit: 1) a smaller data range leads to smaller global sensitivity in DP and consequently to smaller perturbations and better accuracy; 2) a smaller range can be represented using a fewer bits data type, hence reducing the data communication size to reduce fraud detection latency.

Furthermore, the distribution of vertex values evolves dynamically across graph processing iterations. For example, in a PageRank analysis of the Twitter graph, rank scores initially cluster around 0.001 (with a standard deviation of 0.024) during the second iteration. By the sixth iteration, the mean value increases to 0.01, accompanied by a rising standard deviation of 0.065, reflecting both growth and dispersion in the distribution. This highlights that the quantization approach should be adaptive to the distribution changes of vertex values across iterations, in order to minimize accuracy loss.

2) **Adaptive Non-Uniform Quantization:** To accurately represent the highly skewed centrality measures in power-law graphs, CFDGraph employs non-uniform quantization. This is formulated as a 1D K-means clustering problem, optimizing K centroids that best represent quantized values. The clustering is performed *per iteration* on inter-institution data to dynamically adapt to changing data distributions. For privacy, each institution independently performs clustering on its outgoing messages, generating lower-bit representations.

The quantization of messages from institution i to j at iteration t proceeds as follows: **1) Centroid Initialization:** This step is crucial to the clustering quality. To ensure clustering quality while minimizing latency, CFDGraph reuses centroids from the $(t-1)$-th iteration to initialize the t-th iteration, leveraging the dependency between data in two successive graph iterations. For the initial iteration ($t = 0$), K-means++ [34] is used. This approach drastically reduces clustering overhead without compromising quality, making it negligible compared to the total graph processing latency (e.g., reducing the overhead from 300s to under 10s per iteration for PageRank on the Twitter graph). **2) Centroid Assignment and Update:** Given the initial centroids, data points are assigned to the nearest of the K centroids. The centroids are then recalculated based on their assigned points. This assignment-and-update loop repeats until convergence or for a predefined maximum number of iterations. In CFDGraph, we set the maximum number of iterations to five.

The quantization process outputs K centroids and n corresponding centroid IDs, mapping each 32-bit floating-point message to a $\log_2 K$-bit cluster ID in the range [0, K-1].

Algorithm 1: Differentially Private Quantization

Input: D: dataset, d: number of dimensions, $[-r, r]$: dataset range, K: number of clusters, t: maximum number of clustering iterations, ϵ_c: privacy budget for centroid protection
Output: Cluster centroids $\langle o^1, o^2, \ldots, o^K \rangle$

1 $\{o^1, o^2, \ldots, o^K\} \leftarrow$ `InitCentroid`(D);
2 $\epsilon' \leftarrow \frac{\epsilon_c}{(dr+1)t}$;
3 **while** *iter* $\leq t$ **do**
4 **for each** $j \in \{1, 2, \ldots, K\}$ **do**
5 $C^j \leftarrow \{x^\ell : \|x^\ell - o^j\| \leq \|x^\ell - o^i\|, x^\ell \in D, \forall 1 \leq i \leq K\}$;
6 $\langle o_1^j, o_2^j, \ldots, o_d^j \rangle \leftarrow$ `NoisyCentroidUpdate`(d, C^j, ϵ');
7 iter++;
8 **return** $\{o^1, o^2, \ldots, o^K\}$;
9 **Function** `NoisyCentroidUpdate`(d, C, ϵ) **:**
10 Define $\Pi_{[-r,r]}(x) = \begin{cases} -r & \text{if } x < -r \\ x & \text{if } -r \leq x \leq r \\ r & \text{if } x > r \end{cases}$
11 $count \leftarrow |C| + \text{Lap}\left(\frac{1}{\epsilon}\right)$;
12 **for each** *dimension* $i \in (1, 2, \ldots, d)$ **do**
13 $sum_i \leftarrow \sum_{x^\ell \in C} x_i^\ell + \text{Lap}\left(\frac{1}{\epsilon}\right)$;
14 $o_i \leftarrow \Pi_{[-r,r]}\left(\frac{sum_i}{count}\right)$;
15 **return** Cluster centroids $\langle o_1, o_2, \ldots, o_d \rangle$;

This reduces the total communication size from $32n$ bits to $32K + n \log_2 K$ bits, achieving a compression ratio of $32/\log_2 K$ ($n \gg K$).

The value of K is critical for balancing the quantization error and clustering overhead. Figure 4 shows the precision of using the PageRank algorithm on the Friendster graph to identify top-1% highest rank vertices when quantizing inter-institution messages using K values. When K increases, both precision and overhead increase. When K becomes sufficiently large (e.g., $\geq$16), precision stabilizes, but the overhead continues to increase significantly. The optimal K depends on dataset size, distribution skew, and communication costs. For example, the more skewed the data distribution, a larger K is needed. Based on extensive experiments, we selected $K = 16$ as it provides an optimal balance between high accuracy, manageable overhead, and a favorable compression ratio.

3) **Differentially Private Quantization:** To preserve privacy for quantized inter-institution messages, we implement two-stage DP protection, including *centroid protection* and *message value protection*. Denote the privacy budget allocated to each institution per-iteration as ϵ_1. We split the budget into $\epsilon_c = \epsilon_1/2$ for centroids and $\epsilon_{mv} = \epsilon_1/2$ for message values.

Centroid Protection: Algorithm 1 illustrates the flow of the centroid protection stage, which extends an existing privacy-preserving K-means algorithm named DPLloyd [35] for distributed graph setting. Specifically, we first initialize the centroids as described in Algorithm 1 (Line 1) and allocate the privacy budget for different clustering iterations and data dimensions (Line 2). Here, d is the dimension of each data point, r is the range of values and t is the maximum number of iterations in K-means. In each clustering iteration, we assign each data point x^l to the nearest cluster C^j (Line 3-5) and update centroid o^j using a DP-preserved function `NoisyCentroidUpdate` (Line 6). This function works in the following way. The cluster count is first perturbed (Line 11). For each dimension i, the sum of data points in the cluster is perturbed (Line 13) and the centroid coordinate o_i is updated using the perturbed sum and count (Line 14). Algorithm 1 outputs K privacy-protected centroids (Line 9).

Message Value Protection: K-means clustering quantizes message values as centroid IDs (i.e., $\log_2 K$-bit identifiers). To protect quantized values, we perturb them with Laplace noise. The privacy budget ϵ_{mv} is first allocated per remote institution and then per individual message, resulting in a final budget of $\frac{\epsilon_{mv}}{(M-1)n}$ for each message, where M is the number of institutions and n is the number of messages between a pair of communicating institutions. However, since n is typically very large in large-scale CFD, the noise added under this fine-grained budget is substantial. This often corrupts the centroid ID mapping and leads to significant accuracy degradation.

C. Accuracy-Aware Noise Mitigation

While our differentially private quantization protects message privacy, the introduced noise harms fraud detection accuracy. This is exacerbated in large-scale networks where millions of messages cause the per-message privacy budget to become impractically small. To mitigate this noise, we introduce two approaches: enhancing the effective privacy budget and reducing global sensitivity.

1) **Enhancing Privacy Budget with Combiners:** To mitigate the excessive noise from an impractically small per-message privacy budget ($\frac{\epsilon_{mv}}{(M-1)n}$), we introduce *range-partitioned combiners* that exploit the skewed distribution of risk scores (i.e., message values) to increase the privacy budget assigned to each message.

Specifically, we introduce combiners to group n inter-institution messages into n_c partitions ($n_c \ll n$) based on value ranges. To leverage the highly skewed power-law distribution of risk scores, we allocate the privacy budget *non-uniformly*, favoring partitions with smaller values that contain the majority of messages. Using n_c=6 and a value range of $[0, 1]$ as an example, we apply a Fibonacci sequence (1,1,2,3,5,8) to define the partitions (e.g., $[0, \frac{1}{20})$, $[\frac{1}{20}, \frac{2}{20})$, $\cdots$, $[\frac{12}{20}, 1]$ of the value range). The privacy budget is allocated inversely to these weights, granting the combiner for the smallest values (i.e., $[0, \frac{1}{20})$) a large portion ($\frac{8}{20}$ of $\frac{\epsilon_{mv}}{M-1}$), while the combiner for the largest values receives a small portion ($\frac{1}{20}$ of $\frac{\epsilon_{mv}}{M-1}$). This strategy concentrates the budget where most data, with small values, resides, drastically increasing the signal to noise ratio in critical high-density partitions.

The choice of the Fibonacci sequence is motivated by its exponential growth rate ($F_n \approx \phi^n$), which naturally approximates the power-law decay observed in transaction graph degree distributions. We also explored alternative schemes such as Power-of-Two (PoT) quantization [36]. We adapted it for non-negative ranges: $Q^P(\alpha, b) = \alpha \times \{0, 2^{-2^b+1}, 2^{-2^b+2}, ..., 2^{-1}, 1\}$, where α is the value range and b is the logarithm value of n_c. As shown in Figure 6,

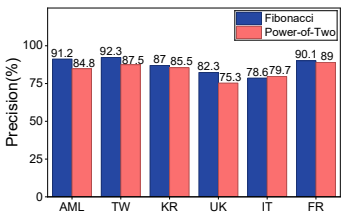


Fig. 6: Precision of PoT vs. Fibonacci strategy

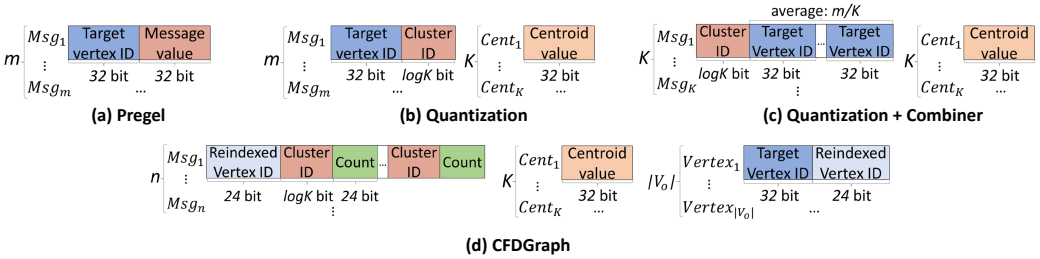


Fig. 7: Inter-institution message format in (a) Pregel; (b) Pregel + quantization; (c) Pregel + quantization + combiners; and (d) CFDGraph.

the Fibonacci-based approach consistently outperforms PoT, leading to our selection of Fibonacci strategy in CFDGraph.

For messages assigned to the i-th combiner, we perform the following steps to perturb the combined message:

1) Sum the float-point values of all messages assigned to this combiner and perturb the sum by adding noise sampled from $Lap(\frac{r+1}{\epsilon})$, where ϵ is the privacy budget assigned to the current combiner. Namely, $Sum_i = Lap(\frac{r+1}{\epsilon}) + \sum_{x \in Combiner_i} x$.
2) Calculate the perturbed average value for the combiner using Sum_i and the message count, namely $Avg_i = \frac{Sum_i}{msg\_count_i + Lap(\frac{r+1}{\epsilon})}$.
3) Select a cluster ID for the entire combiner according to the average value, namely $ID_i = GetIndex(Avg_i)$.

After these three steps, a cluster ID is selected for each combined message. When the target institution receives a message, it dequantizes the message to the perturbed centroid value of the corresponding cluster. This value is then sent to all receiving vertices in the target institution for the next iteration of graph processing.

2) **Reducing Global Sensitivity with Clipping:** The effectiveness of our combiners is limited by the skewed message value distribution, where the long-tail distributions still dominate global sensitivity. To complement the combiner design, we introduce range clipping to reduce global sensitivity by constraining extreme values:

$$Clip(x) = \max(lb, \min(ub, x)) \tag{2}$$

where lb and ub respectively denote the lower and upper clipping bounds.

As discussed in Section IV-B, graph messages follow a power-law distribution, meaning that a few maximum values occupy a wide range of values. Therefore, applying a clipping technique can effectively reduce the global sensitivity, thereby decreasing the added noise and improving system usability. After applying clipping, the formula for computing the sum of each cluster in Algorithm 1 becomes $sum_i = Lap(\frac{1}{\epsilon}) + \sum_{x^l \in C} Clip(x_i^l)$. Clipping is also applied to the combiners when calculating the sum of values.

The clipping bounds are crucial for the effectiveness of fraud detection: an upper bound that is too high introduces excessive DP noise, while an upper bound that is too low introduces inaccuracies due to data loss. We bound the values to $[0, r]$, where $r = ub - lb$, which aligns with transaction semantics (no negative risk values). Figure 5 illustrates the impact of different upper bounds r on the precision of the PageRank algorithm on the Friendster graph [37]. When r is greater than 5, the precision of Pregel is close to 100%, indicating that a small upper bound does not harm the usability of graph algorithms. When DP is integrated, the precision of CFDGraph initially increases with increasing r and then drops sharply as the large noise overwhelms the values. In this example, we select an upper bound of 0.5 for rank values of vertices to achieve good data utility.

D. Message Compression Techniques

In multi-institution fraud detection systems, performance is significantly hindered by the communication overhead of exchanging data across institutions, particularly due to limited global area network (GAN) bandwidth. Recent advances in compressed graph direct computing [38], [39], [40] have shown that compression-aware processing provides a novel and effective way to reduce communication overhead while maintaining computational efficiency. However, these techniques (such as our k-means quantization) reduce only the size of transmitted message *values*, while the cost of transmitting message *meta-data* (i.e., target vertex IDs) remains substantial.

Consider m messages sent from institution i to j. Figure 7 illustrates the format of the messages in the traditional graph processing engine Pregel [22] and after applying the optimizations in CFDGraph. As shown in Figure 7(a), each message comprises a 32-bit floating-point value (representing the risk score being propagated) and a 32-bit integer (identifying the *target vertex ID* in Pregel). We applied techniques such as Quantization and Combiner in CFDGraph, and the updated message formats are illustrated in Figure 7(b) and (c). For example, Figure 7(b) illustrates that, our K-means quantization replaces message values using cluster IDs, which reduces the size of each message value from 32 bits to $\log_2 K$ bits. An additional message is required to transmit centroid values (K 32-bit values). Furthermore, combiners group multiple messages into a single one in the format shown in Figure 7(c). Each combined message contains one cluster ID ($\log_2 K$-bit) and multiple target vertex IDs representing the destinations of the messages in this group. From the examples, we can conclude that quantization and combiners compress the *message values*, but target *vertex IDs* still dominate the communication cost.

To address this bottleneck, we introduce two complementary techniques: 1) *vertex reindexing* to reduce the bit-length of vertex ID representation, and 2) *message aggregation* to minimize redundant transfers of identical vertex IDs.

Vertex Reindexing. To reduce the bit-length of vertex IDs, we reindex vertices using smaller data types. Specifically, for each institution pair i to j, we identify vertices in j with inter-institution neighbors in i and reindex these vertices by sequentially reassigning shorter IDs. For example, for a Bitcoin-OTC graph distributed in five institutions (details in Section VI), reindexing reduces average vertex ID size from 32 bits to 12 bits, requiring only a one-time preprocessing step. To help mapping reindexed vertex IDs back to the original IDs after arriving at institution j, we additionally send the <original ID, reindexed ID> mapping from institution i to j, as shown in Figure 7(d). Note that these mappings only need to be sent once between each pair of institutions.

Message Aggregation. Among the graph processing messages sent from institution i to j, more than one may end up at the same target. Traditional systems redundantly send the same target vertex ID multiple times. To mitigate this, we aggregate messages bound for the same vertex as illustrated in Figure 7(d). n distinct target vertices result in n aggregated messages. Each aggregated message adopts the format <vertex ID, $index_1$, $count_1$, $index_2$, $count_2$, ... >, where $index$ denotes the cluster ID and $count$ reflects the number of messages with value $index$. We theoretically analyze the effectiveness of our message compression techniques in Section V.

E. Putting It All Together

Two financial institutions (A and B) collaboratively process transaction data using the following steps:

- **Step 1: Pre-communication Centroid Exchange**. Each institution *independently* runs local K-means clustering on its own message values (Algorithm 1). After this, the perturbed float-point centroid values are exchanged. For example, Inst. A (K=16) sends its 16 centroids $(C_{a,0}, ..., C_{a,15})$ to Inst. B, and Inst. B (K=2) sends its 2 centroids $(C_{b,0}, C_{b,1})$ to A.
- **Step 2: Quantized Transmission**. Each message to be sent between A and B is replaced with the index of its closest centroid (message value protection in Section IV-B3). A sends 4-bit indexes (e.g., index 5 as the closet centroid) to B, while B sends 1-bit indexes (e.g., index 0 as the closet centroid) to A, greatly reducing inter-institution communication size. Applying combiner technique in Section IV-C1, inter-institution messages are further grouped and the message format is modified as in Figure 7.
- **Step 3: Dequantization**. Upon receiving an index, the recipient replaces it with the corresponding 32-bit centroid value. For example, B decodes the message value "5 " (received from A) to $C_{a,5}$ and A decodes the message value "0" to $C_{b,0}$. All subsequent computations (e.g., PageRank updates) use these dequantized values to ensure high precision.

TABLE I: Privacy budget notations

Symbol	Definition and Description
ϵ_{total}	Total privacy budget to entire application
ϵ_1	Budget assigned to an inst. per iteration, $\epsilon_1 = \frac{\epsilon_{total}}{M*T}$
ϵ_c	Budget for centroid perturbation, $\epsilon_c = \epsilon_1/2$
ϵ_{mv}	Budget for message values perturbation, $\epsilon_{mv} = \epsilon_1/2$

V. Theoretical Analysis

A. Privacy Analysis

Theorem 1. *The CFDGraph system satisfies ϵ-edge-DP against possibly colluding honest-but-curious institutions.*

Proof. The graph computation in CFDGraph runs for T fixed iterations with identical execution. We prove ϵ-edge-DP by showing: (1) all data-dependent computations per iteration are perturbed under edge-DP, (2) total privacy consumption is bounded by ϵ via composition, and (3) all Laplace mechanisms use correct sensitivity and budget parameters.

First, point (1). Consider an arbitrary institution and assume a single remote institution. The extension to multiple remote institutions is straightforward. Each iteration performs three data-dependent steps: (i) *Centroid Computation*: Algorithm 1 perturbs counts (Line 11, sensitivity $\Delta = 1$) and sums (Line 13, sensitivity $\Delta = r$) using the Laplace mechanism to get perturbed centroids, satisfying edge-DP. (ii) *Message Averaging* at Combiners: For each combiner CB_i in a set, the average $\text{Avg}_i = \frac{\sum_{x \in \text{CB}_i} x + \text{Lap}(\Delta/\epsilon)}{|\text{CB}_i| + \text{Lap}(\Delta/\epsilon)}$ uses Laplace noise with sensitivity $\Delta = r + 1$ (due to clipping) and edge-DP. ϵ is explained in point (2). (iii) *Closest Centroid Identification*: Operates solely on outputs from (i) and (ii). By post-processing immunity (Proposition 2.1 [41]), it satisfies edge-DP without additional noise. **Thus, all data-dependent steps are edge-DP perturbed.**

Second, point (2). Let ϵ_{total} be the total privacy budget. ϵ_{total} is allocated across T iterations and M institutions as $\epsilon_c = \epsilon_{\text{mv}} = \frac{\epsilon_{\text{total}}}{2MT}$ for centroids (ϵ_c) and message averaging (ϵ_{mv}): (i) *Centroids*: Budget ϵ_c covers $(r+1) \cdot t$ queries to sum and count per institution, each allocated $\frac{\epsilon_c}{(r+1)t}$ (sequential composition). (ii) *Message Averaging*: For $M-1$ remote institutions, ϵ_{mv} is split per set of combiners. Combiner i uses budget $\frac{w_i \epsilon_{\text{mv}}}{M-1}$ with $\Delta = r + 1$ (vector release of (sum, count)), where w_i is its Fibonacci weight, ensuring $\sum_i w_i = 1$ (sequential composition). Total iteration budget per institution is $\epsilon_c + \epsilon_{\text{mv}} = \frac{\epsilon_{\text{total}}}{MT}$. Summing over T iterations and M institutions gets ϵ_{total}. **Thus, the privacy budget spent during graph computation does not exceed ϵ_{total}.**

Third, point (3). All mechanisms use correct sensitivities and budgets: (i) *Centroids*: For counts, $\Delta = 1$ and for sums, $\Delta = r$. Budget for each centroid query is $\frac{\epsilon_c}{(r+1)t}$, as shown in Algorithm 1. (ii) *Message Averaging (per combiner i)*: for the vector $(\text{sum}, \text{count})$, $\Delta = r + 1$ and budget is $\frac{w_i \epsilon_{\text{mv}}}{M-1}$. **Thus, all Laplace mechanisms are properly parameterized.**

Points (1)–(3) collectively ensure ϵ-edge-DP. □

B. Utility Analysis

In this section, we analyze the trade-off between privacy and utility using the PageRank [42] algorithm from two aspects.

1) **Differentially private quantization:** The utility loss from this aspect stems from two sources: centroid perturbations and quantization approximation.

For centroid perturbation, referring to [35], Normalized Intracluster Variance (NICV) is usually used to evaluate the quality of K-means clustering, and K-means can be considered to have converged when NICV reaching a certain threshold.

Lemma 1. *Given n data points and o is one of the centroids after applying K-means on the data points. Denote $\hat{o}$ as the perturbed centroid corresponding to o obtained using Algorithm 1. We have [35]*

$$|\mathrm{NICV}(\hat{o}) - \mathrm{NICV}(o)| \leq \|\hat{o} - o\|^2$$

We use the *mean squared error* (MSE) between o and $\hat{o}$ to approximate the utility loss introduced by perturbation:

$$\mathit{MSE}_{ct} = \mathbb{E}\left[|\mathrm{NICV}(\hat{o}) - \mathrm{NICV}(o)|\right] = \mathbb{E}\left[\|\hat{o} - o\|^2\right] \quad (3)$$

To analyze the scale of MSE, we assume two conditions: 1) the number of dimensions of the data is 1; 2) the noise $\Delta|C|$ is much less than $|C|$. We have the following lemma.

Lemma 2. *For one centroid in one iteration. Let r be the range of values, ϵ be the privacy budget, $|C|$ be the size of cluster C, t is the iteration of K-means, $|D|$ be the size of the message set per institution and K be the number of clusters. Then, the error introduced by DP noise can be estimated as:*

$$\mathit{MSE}_{ct} \approx 2(1+r^2)\left(\frac{(r+1)t}{\epsilon|C|}\right)^2 \approx 2(1+r^2)\left(\frac{(r+1)tK}{\epsilon|D|}\right)^2 \quad (4)$$

Empirical validation. Set r to 0.5, t to 5, ϵ to 10^{-2} corresponding to ϵ_c in Table I, and $|C|$ to 10^6. The magnitude of MSE_{ct} is thus 10^{-6}. For a billion-edge graph (e.g., Table II), our evaluation has shown that the centroid values range from 10^{-3} to 10^{-1}, making MSE_{ct} negligible.

Quantization introduces bias by representing messages with centroids.We evaluate this using distortion, defined as $d(x, \overline{x}) = |x - \overline{x}|^2$, where x is the dequantized value and $\overline{x}$ is the original value [43]. For a single centroid in one iteration, we define the mean distortion as MSE_{qt}:

$$\mathit{MSE}_{qt} = \mathbb{E}[d(\hat{o}, x)] = \frac{1}{|C|}\sum_{x^\ell \in C}|\hat{o} - x^\ell|^2 \approx \frac{K}{|D|}\sum_{x^\ell \in C}|\hat{o} - x^\ell|^2 \quad (5)$$

For top-k selection, we do not need to calculate the exact value of the rank, so this part of the loss is acceptable.

2) **Combiners:** Since the way of perturbation in Combiner is similar to centroid perturbation, we use the MSE in Equation 3 to estimate utility loss:

$$\mathit{MSE}_{cb} = \mathbb{E}\left[\|\hat{Avg} - Avg\|^2\right] \quad (6)$$

where $\hat{Avg}$ and Avg are the noisy average and original average. Furthermore, we can use the same derivation method to finally obtain the following result:

$$\mathit{MSE}_{cb} \approx 2(1+r^2)\left(\frac{r+1}{\epsilon \cdot msg\_count}\right)^2 \quad (7)$$

Empirical validation. Set r to 0.5, ϵ to 10^{-4} (a portion of $\frac{\epsilon_{mv}}{M-1}$), and msg_count to 10^6. The magnitude of MSE_{cb} is thus 10^{-4}. Compared to Avg that ranges from 10^{-3} to 10^{-1}, this error is trivial.

C. Communication Analysis

CFDGraph reduces communication through two techniques from Section IV-D: vertex reindexing and message aggregation. The communication size from institution i to j can be analyzed as follows.

Denote V as the set of vertices in institution j and V_o as the smallest subset communicating with institution i. Before optimization, the size of messages is $m * (log|V| + 32)$ bits, where m is the number of messages. After optimization, the size is reduced to $n * (log|V_o| + 2 * (logK + 24))$ bits on average, where n is the reduced number of messages after aggregation and K is the number of centroids. Each target has two different indexes on average, and 24 is the bit width of the count in Figure 7(b) that $24 * logK$ suffices to represent graphs with vertex size of 10^8.

In summary, the communication reduction rate of CFDGraph compared to the baseline Pregel can be calculated as:

$$\frac{m * (log|V| + 32)}{n * (log|V_o| + 2logK + 48)} \quad (8)$$

Empirical validation. Consider an institution with $|V| = 10^7$. $|V_o|$ is typically $0.8|V|$ and K is 16. The average vertex degree is approximately 15 (i.e., $m = 15$) and the communication size is reduced by around 11x compared to Pregel.

To quantify the overhead of transmitting Vertex ID mappings, we model the cost as follows. Let $|V_{all}|$ be the total vertices in the global graph and M be the number of institutions. The average partition size is $|V_{avg}| = |V_{all}|/M$. In the worst-case scenario (all-to-all communication), there are $M(M-1)$ channels. Since each target ID is compressed to a 24-bit reindexed value alongside its 32-bit original ID, the total synchronization overhead (GAN_{re}) is:

$$GAN_{re} \approx (32+24) \times M(M-1) \times |V_{avg}| \approx 56(M-1)|V_{all}| \quad (9)$$

This demonstrates that the overhead scales linearly with the coalition size M for a fixed total graph size. Empirically, on the Friendster graph ($|V_{all}| \approx 66M$), the recurring communication for risk score propagation during graph processing is 43GB, while the one-time index synchronization requires only $\sim$1GB, showing negligible overhead ($< 2.5\%$) in practice.

D. Complexity Analysis

We analyze the time complexity of the core components in CFDGraph to demonstrate its scalability. Let n be the number of inter-institution messages, K the number of quantization centroids, and I the number of graph processing iterations.

1) Adaptive Quantization: The complexity is dominated by the 1D K-means clustering performed in each iteration. With t clustering iterations and dimension $d = 1$, the cost is $O(nKdt)$ per graph iteration. Over the full execution, the total complexity is $O(nKdtI)$. Since K (e.g., 16), d, and t (e.g., 5) are small constants, the overhead scales linearly with n.

2) Combiner and Clipping: Each message is processed once per iteration. The Combiner maps values to partitions

TABLE II: Evaluated Large Graphs

Graph	AML	TW	KR	UK	IT	FR
Vertices	27M	42M	34M	39M	41M	66M
Edges	38M	1468M	1174M	936M	1151M	1806M

TABLE III: Evaluated Small Graphs

Graph	BA	BO	ASb	ASu	AWH	AWL
Vertices	4k	6k	100k	100k	515k	706k
Edges	24k	36k	2M	2M	5M	7M

in $O(n)$ and aggregates statistics in $O(n_c)$ where n_c is the number of combiners ($n_c \ll n$). Thus, the total complexity is $O(nI)$, which is linear with respect to the graph size.

3) Vertex Reindexing: This technique is applied as a one-time preprocessing step. It involves sorting vertices by degree or ID, incurring a worst-case complexity of $O(V \log V)$, where V is the number of boundary vertices. In dynamic sliding-window scenarios, incremental updates can further reduce this cost toward amortized linear time.

In summary, the dominant computational components of CFDGraph scale linearly with the dataset size, ensuring efficiency for large-scale collaborative fraud detection.

VI. Evaluations

A. Experiment Settings

Testbed. We implemented CFDGraph on a computing cluster comprising 6 nodes. Each node is equipped with a 64-Core AMD EPYC 7742 CPU and 1 TB of memory. To simulate the constrained bandwidth typical of cross-institution networks, the nodes are interconnected via a standard 1 Gbps LAN.

Dataset. We evaluate CFDGraph using six real-world large graphs as shown in Table II and six small financial graphs in Table III. AML is a Anti-Money Laundering (AML) graph mimicking Alipay's transaction network [44]. To our knowledge, it is the largest open financial graph available. Due to the scarcity of open-sourced *large* financial graphs, we supplement this with five additional large-scale social/web graphs, including Twitter (TW), Kron (KR), uk-2005 (UK), it-2004 (IT) and Friendster (FR) [45], [46], [47]. Bitcoin-Alpha (BA) and Bitcoin-OTC (BO) are two transaction graphs used in bitcoin trading [48]. AMLSim-bal (ASb), AMLSim-unb (ASu), AMLWorld-HI (AWH), AMLWorld-LI (AWL) are four synthetic datasets that simulate money laundering transactions [44], [49], [50].

The above datasets do not contain ground truth. To detect fraud, we run four popular random-walk graph algorithms to infer anomolies as ground truth, including PageRank (PR) [42], Personalized PageRank (PPR) [12], HITS [24] and SALSA [51]. The global sensitivity of PageRank and Personalized PageRank after clipping is 0.5 while the global sensitivity of HITS and SALSA are both 1. All graph algorithms execute a fixed number of iterations, which is set to 10 by default.

Comparisons. We compare CFDGraph with the following state-of-the-art graph processing systems.

- **Pregel-DP**. This is the Pregel system integrated with naive DP technique to protect privacy. It uses the same DP functions as CFDGraph, and perturbs inter-institution messages without quantization/combiner/clipping optimizations.
- **PGPregel** [13] is a privacy-preserving systems for processing large-scale data distributed in multiple data centers. It trades-off graph processing accuracy, privacy, and performance, and is considered as the state-of-the-art comparison.
- **RAGraph-DP**. RAGraph [21] is a distributed framework specifically designed for graph processing across multiple regions, incorporating several techniques to reduce inter-region data communications. Similar to Pregel-DP, we add differential noise to the inter-region messages for privacy.

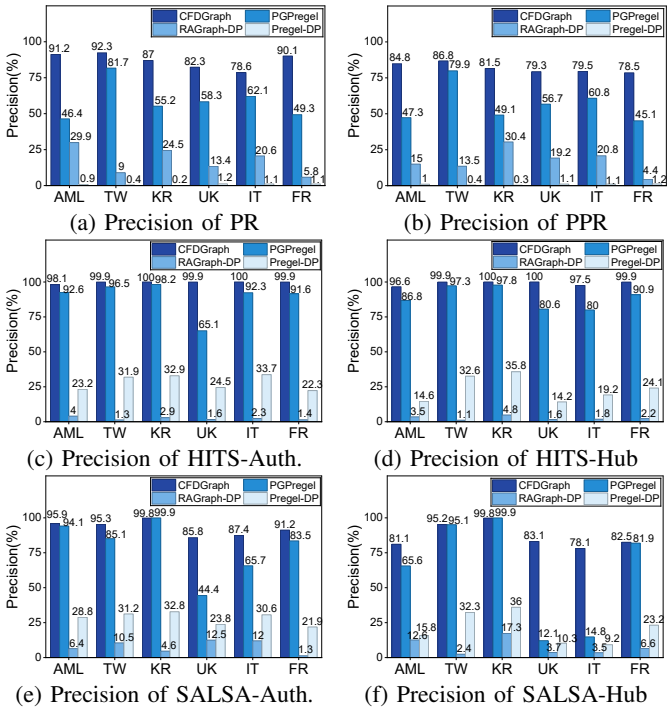
Our evaluation focuses on comparing CFDGraph against distributed graph processing systems, as they represent the most *scalable and practical* class of approaches for industry-scale financial fraud detection. Existing other methods like GNN-based fraud detection [52] and private graph synthesis [53], they are unsuitable as direct baselines for this work due to either requiring offline model training and hence cannot adapt to dynamic graph changes, or have prohibitive computational costs (e.g., [52] fails to process our smallest graph in 24 hours).

Metrics. We evaluate the accuracy and recall of the compared systems in detecting the top-k most risky accounts, where the true top-k set is obtained by calculating the risk scores of vertices using naive Pregel (no DP noise). Specifically, the *accuracy* is calculated using the precision of the top-k retrieved vertices, namely $P = \frac{TP}{TP+FP}$, where TP are the frauds correctly identified in the retrieved top-k set and FP are those retrieved but are not actual frauds. Similarly, *recall* is calculated as $R = \frac{TP}{TP+FN}$, where FN are the actual frauds absent from the retrieved top-k.

Achieving 100% recall is crucial for fraud detection to prevent financial losses. To address this, we propose the *100% recall size (S_R)* metric. It measures the size of the top-k set needed to achieve 100% recall, expressed as multiples of k. For example, if an algorithm needs to retrieve 1000 nodes to cover all the actual top-100 nodes, then S_R is 10 (i.e., 1000/100).

We compare the latency of different systems to evaluate their timeliness in identifying fraudulent nodes. Due to the limited inter-institution network bandwidth, inter-institution data communication time becomes the main performance bottleneck. We denote such inter-institution network communication as Global Area Network (GAN) communication and use the GAN usage as a measure of system latency.

Configurations. We simulate a multi-institution environment with M institutions. By default, $M = 5$. We distribute graphs to institutions based on the real-world geographic distribution of Twitter users [13]. Specifically, the proportions of vertices in the five institutions are 43%, 31%, 11%, 10%, and 5%. We randomly assign vertices to different institutions according to the specified proportions, resulting in different proportions of inter-institution edges for different graphs (from 55% to 80%). The privacy budget is an important parameter for DP and by default $\epsilon = 1$. For the top-k selection, we set k to 1% by default. There are several important parameters in CFDGraph, such as the number of clusters in K-means



(a) Precision of PR (b) Precision of PPR

(c) Precision of HITS-Auth. (d) Precision of HITS-Hub

(e) Precision of SALSA-Auth. (f) Precision of SALSA-Hub

Fig. 8: Precision results on the six large real-world graphs

quantization and the number of combiners. We empirically select the best value for these parameters in the experiments.

We perform six sets of sensitivity studies in Section VI-C to evaluate the impact of top-k sizes, different privacy budgets, number of institutions M, number of combiners, the amount of inter-institution edges, and graph sizes on the effectiveness and efficiency of CFDGraph. Unless stated otherwise, the graph used is Friendster and the graph algorithm is PageRank.

B. Overall Results

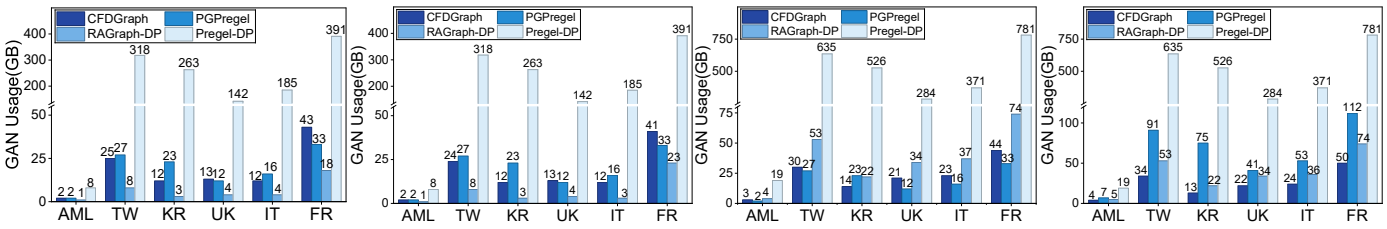
Figures 8, Figure 9 and Table IV present the accuracy, GAN usage and recall results of four scalable graph processing systems on six large-scale graphs.

1) **Accuracy Results:** Comparing the accuracy results in Figure 8, we have the following observations. *First*, CFDGraph achieves the best precision under all settings. Specifically, CFDGraph improves the top-1% fraud detection precision by 1%-71%, 51%-99% and 57%-92% compared to PGPregel, RAGraph-DP and Pregel-DP, respectively. *Second*, CFDGraph performs especially well on HITS and SALSA. We differentiate their precision by Authority (Figure 8 (c) and (e)) and Hub (Figure 8 (d) and (f)). For both authorities and hubs, CFDGraph can obtain very high precision (up to 100% on HITS and 99.9% on SALSA), demonstrating its capability of effectively detecting different fraud patterns (e.g., fan-in and fan-out). The reason that CFDGraph performs especially well on these two graph algorithms is because of their limited value range (i.e., [1, 2] for HITS and [0, 1] for SALSA), which results in smaller global sensitivity and smaller DP noise. *Third*, for PR and PPR algorithms, although CFDGraph can hardly reach $> 90\%$ precision, it still achieves a large lead in the precision results compared to the other three systems. Especially, compared to PGPregel, which is considered as the SOTA privacy-preserving graph engine, CFDGraph increases the precision by 10.6%-44.8% on PR and 6.9%-37.5% on PPR. *Finally*, one interesting finding is that, the precision results of RAGraph-DP are even worse than the baseline Pregel-DP in many cases (e.g., Figure 8 (c)-(f)). This is because RAGraph adopts a delta-based model when processing graph algorithms. Instead of sending the original message, it only transmits a message when the delta between the current message and the previous one exceeds a specified threshold. While this approach significantly reduces the size of data communication, the messages become very small in value and the influence of noise overshadows the messages' semantics.

2) **Latency Results:** Comparing the GAN usage results in Figure 9, we have the following observations. *First*, all of CFDGraph, PGPregel and RAGraph can effectively reduce the inter-institution data communication size compared to the baseline Pregel-DP. Specifically, CFDGraph reduces the GAN usage by 75%-98% compared to Pregel-DP. *Second*, RAGraph-DP aggregates messages by vertices and adopts a message filtering strategy to send messages only when they have accumulated certain importance. This explains why RAGraph-DP achieves the lowest GAN usage on PR and PPR. However, for HITS and SALSA, the message filtering strategy is disabled since it does not meet the monotonicity requirement of the filtering technique. Therefore, CFDGraph obtains lower GAN usage than RAGraph-DP in these two cases (Figure 9 (c) and (d)). *Third*, PGPregel obtains relatively lower GAN usage for PR, PPR and HITS. This is because PGPregel adopts a sampling technique to filter inter-institution messages, which is a trade-off between accuracy and performance. Since the sampling rate for SALSA is 100%, PGPregel generates higher GAN usage than CFDGraph and RAGraph-DP (Figure 9 (d)).

3) **Recall Results:** Table IV shows the S_R results of CFDGraph and PGPregel. Since the precision of RAGraph-DP and Pregel-DP are much lower than CFDGraph, we have omitted their S_R values in the table. The S_R metric measures the size of the retrieved top-k set relative to the size of the actual top-k set when achieving 100% recall. We vary k from 0.01%, 0.1% to 1%. The results show that CFDGraph can greatly reduce the size of the retrieved data set compared to PGPregel. For example, for the Twitter graph and the PPR algorithm, the top-0.1% set retrieved by CFDGraph to reach 100% recall is only $\frac{1}{78}$ of the size of the top-0.1% set retrieved by PGPregel. For HITS, CFDGraph achieves a S_R of 1 for all three k values, meaning that it can obtain 100% recall in all cases. This makes CFDGraph especially useful in practice.

In summary: The results with large financial and social graphs demonstrate that 1) CFDGraph is effective in identifying frauds in large-scale graphs, achieving high accuracy (100% precision for HITS fraud detection algorithm); 2) CFDGraph promises high efficiency for fraud detection, where the small GAN usage (12GB-50GB) keeps inter-institution data communication overhead low (6s-25s per iteration assuming 10MB/s GAN bandwidth); and 3) CFDGraph is practical, achieving 100% recall with a much smaller top-k size.

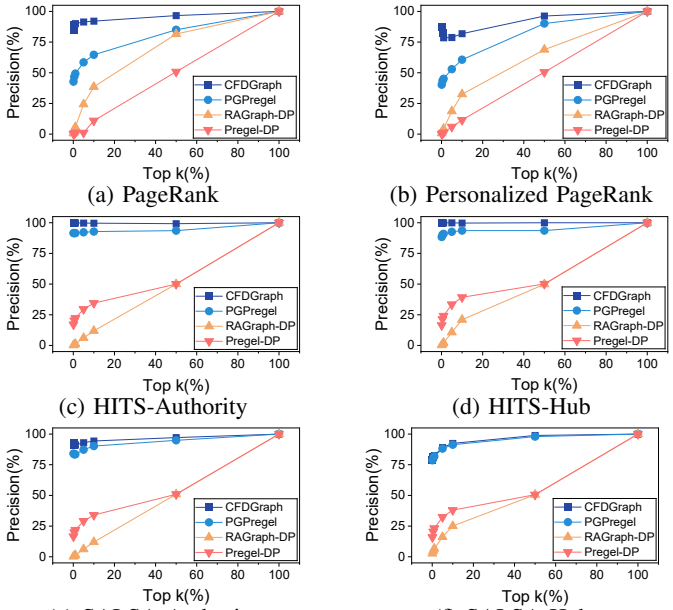


(a) GAN Usage of PR (b) GAN Usage of PPR (c) GAN Usage of HITS (d) GAN Usage of SALSA

Fig. 9: GAN usage on the six large real-world graphs

TABLE IV: S_R for four algorithms on six large graphs using CFDGraph and PGPregel

	Graph	AML			TW			KR			UK			IT			FR		
	Top@K%	0.01	0.1	1	0.01	0.1	1	0.01	0.1	1	0.01	0.1	1	0.01	0.1	1	0.01	0.1	1
CFDGraph	PR	14.6	3.6	21.1	22.7	6.6	43.7	976.5	112.1	11.2	3041	601.2	92.3	1865.2	250.9	93.4	238.8	251.6	39.6
	PPR	9.5	6.3	25.2	3.7	4.4	42.1	3380.4	983.2	99.8	9017.7	933.4	98	9067.1	961.9	99.6	266.8	787.6	95.9
	HITS(Auth.)	1	1	1	1	1	1	1	1	1.1	1	1	1	1	1	1	1	1	1
	HITS(Hub)	1	1	1	1	1	1	1	1	1	1.1	1	1	1	1	1.1	1	1	1
	SALSA(Auth.)	26.8	13.6	10	4.1	49.4	9.5	1	1	1	539.2	241.9	42.7	124	18	45.8	2.2	10.5	34.6
	SALSA(Hub)	13.9	18.1	34.6	27.7	34.1	51.3	1	1	1	352.3	632.6	84.8	730.4	517.6	81.7	140.3	147.6	29.9
PGPregel	PR	1231.4	732.6	95.6	205.6	99.8	94.3	4523.8	872.9	87.9	9836.3	998	100	9953.6	999.9	100	5385.5	987	100
	PPR	804.9	513.5	88.7	191.9	342.7	99.5	10000	1000	100	9764.2	999.1	100	9991.3	999.1	100	9718.7	985.9	100
	HITS(Auth.)	6.8	3.6	4.9	1.2	3.2	1.5	1	1.1	1.4	2.4	23.2	3.3	8.1	2.4	1.8	2.9	2.9	3.1
	HITS(Hub)	8.9	5.5	7.8	1.3	1.3	1.4	1.2	1.1	1.2	68.8	9.8	10.8	2.7	3.5	7.2	3.5	3.0	2.6
	SALSA(Auth.)	73.3	40.7	55.8	37.7	47.1	14.3	1	1	1	323.2	78.8	18.1	89	25.5	17.8	37.1	55.9	51.2
	SALSA(Hub)	108.7	89.4	63.8	46.3	38.4	44.9	1	1	1	83.1	225.4	69.2	301.2	274.4	63	803.2	165.3	28.6



(a) PageRank (b) Personalized PageRank (c) HITS-Authority (d) HITS-Hub (e) SALSA-Authority (f) SALSA-Hub

Fig. 10: Impact of top-k size

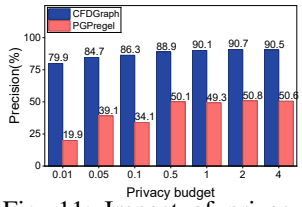


Fig. 11: Impact of privacy budget

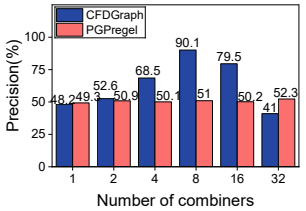


Fig. 12: Impact of number of Combiners

C. Sensitivity Study

1) Impact of Top-k Sizes: Figure 10 shows the precision results under different top-k sizes. Specifically, we vary k from 0.1%, 0.5%, 1%, 5%, 10%, 50% to 100% of the number of vertices using the Friendster graph. We make the following observations. First, CFDGraph achieves the highest precision of all under all settings. It is evident that CFDGraph performs well regardless of the value of k. Second, as k increases, the precision also increases for all compared systems. It is worth noting that, when k is small (e.g., 0.1%), the precision of CFDGraph is always good ($> 75\%$), making it a practical solution. For HITS, the results of CFDGraph remain close to 100% regardless of the value of k.

2) Impact of Privacy Budget: In general, a lower privacy budget provides more protection, but results in more noise. We study its impact on precision by varying the budget from 0.01 to 4 using the Friendster graph and the PageRank algorithm (see Figure 11). As the figure illustrates, CFDGraph achieves higher precision than PGPregel under different privacy budgets. For example, the precision of CFDGraph under a very low privacy budget (i.e., 0.01) is 79.9%, 60% higher than PGPregel.This demonstrates the effectiveness of CFDGraph even with a low privacy budget or a stricter privacy model.

3) Impact of Number of Combiners: The number of combiners presents a trade-off on fraud detection precision: too few combiners increase quantization error from value averaging, while too many amplify DP noise by reducing the budget per combiner. We study the impact of this parameter by varying the number of combiners between each pair of institutions from 1 to 32. As shown in Figure 12, CFDGraph achieves higher precision when the number of combiners increases from 1 to 8, reaching a precision of 90.1% (39.1% higher than PGPregel). However, as the number of combiners increases beyond 8, the precision starts to decrease. Empirically, we set

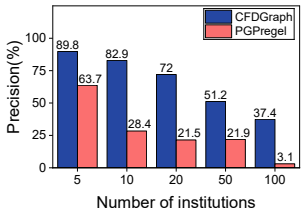


Fig. 13: Impact of number of Institutions

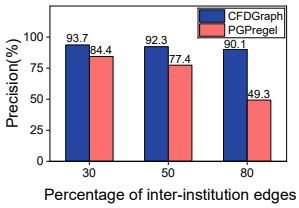


Fig. 14: Impact of inter-institution edge percentage

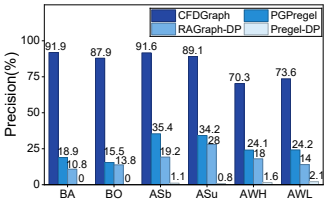


Fig. 15: Precision of small graphs

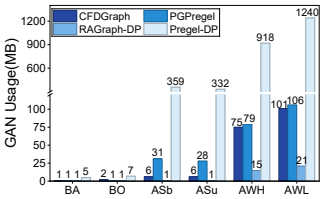


Fig. 16: blue GAN Usage of small graphs

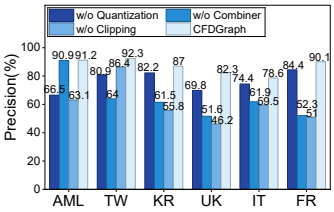


Fig. 17: Precision of PR

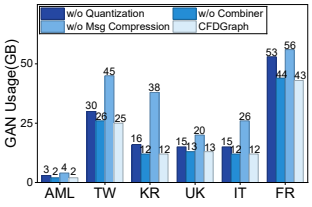


Fig. 18: GAN Usage of PR

the best number of combiners to 8 to achieve good precision.

4) Impact of Number of Institutions: In this experiment, we study the impact of increasing the number of institutions on the effectiveness of CFDGraph. Specifically, we vary the number of institutions from 5, 10, 20, 50 to 100, and distribute vertices evenly among them. Figure 13 shows the results obtained by CFDGraph and PGPregel. As the number of institutions increases, the precision of both CFDGraph and PGPregel decreases. This is because the privacy budget is shared among multiple institutions to protect the privacy of inter-institution messages. When the number of institutions increases, the budget per message decreases, causing a significant drop in precision. Despite this, CFDGraph consistently outperforms PGPregel under all settings, indicating that CFDGraph is more scalable and suitable for larger-scale systems.

In real financial ecosystems, 5–20 institutions is a practical range, as bank branches often function as single entities in CFDGraph. Recent study [44] models collaborative money laundering with only two institutions, one representing e-Commerce platform and the other aggregating all external entities. Further, by processing data in discrete time windows that involve only currently active institutions, CFDGraph naturally limits the number of simultaneous participants.

5) Impact of Inter-institution Edges: In large-scale graph processing, the proportion of inter-institution edges directly influences precision, as it determines the volume of messages requiring noisy perturbation. To study its impact, we vary the proportion from 30%, 50% to 80% for the Friendster graph. Figure 14 shows the results obtained by CFDGraph and PGPregel for the PageRank algorithm. First, CFDGraph obtains higher precision compared to PGPregel under all proportions. Second, the precision of CFDGraph slightly decreases as the proportion of inter-institution edges increases, while the precision of PGPregel declines steeply. CFDGraph's robustness stems from its combiners, which allocate the privacy budget based on the data distribution (e.g., the Fibonacci sequence), making it less sensitive to the number of edges. This design minimizes accuracy loss by averaging aggregated messages.

6) Impact of Graph Sizes (Sliding Windows): As discussed in Section III-A, the transaction graph is dynamically constructed within discrete, non-overlapping sliding windows. Consequently, the window size determines the temporal scope and directly dictates the scale of the constructed graph. Quantitatively, according to [54], Alipay processes transactions at an average rate of thousands of transactions per second (TPS). Based on this estimate, the **AML** graph corresponds to a window of approximately 10.6 hours, while the five larger social graphs correspond to windows exceeding 260 hours.

To evaluate CFDGraph under shorter, real-time window constraints, we additionally tested six smaller financial datasets: AMLSim-bal (ASb, 0.5h), AMLSim-unb (ASu, 0.5h), AMLWorld-HI (AWH, 1.4h), AMLWorld-LI (AWL, 2.0h), Bitcoin-Alpha (BA, 24s), and Bitcoin-OTC (BO, 36s). Figures 15 and 16 present the precision and GAN usage results on these datasets using the PageRank algorithm.

The results demonstrate that CFDGraph significantly outperforms the baselines across all window sizes. Notably, **PGPregel** fails to maintain high accuracy on these smaller, short-window financial graphs. This is because the edge sampling technique employed by PGPregel discards critical structural information when the total graph size is small. In contrast, CFDGraph preserves high precision on both small (short-window) and large (long-window) graphs, confirming its robustness for dynamic fraud detection.

D. Ablation Study

To study the individual contribution of each technique, we evaluated variants of CFDGraph by selectively disabling *Adaptive Quantization*, *Combiners*, *Clipping*, and *Message Compression*. Figure 17 and Figure 18 show the precision and GAN usage results for these variants on PageRank.

Our first observation is that, no single component is sufficient. CFDGraph achieves high precision and low communication overhead only through the joint integration of all techniques. Second, for precision, *Combiner* and *Clipping* contribute more significantly. Specifically, *Combiner* effectively increases the usable privacy budget, as the per-message privacy budget prior to aggregation is extremely small, while *Clipping* reduces the global sensitivity, which would otherwise be very large.Third, for GAN usage, *Message Compression* makes the most significant contribution. Disabling it results in a $2.17\times$ increase in overhead. This improvement primarily stems from *Vertex Reindexing*, which drastically reduces the bits required for target IDs. Furthermore, *Quantization* is critical for bandwidth efficiency, compressing message values from

32 bits to 4 bits. Without these compression techniques, the system would exceed practical bandwidth constraints.

VII. Related Work

Graph-Based Fraud Detection. Existing approaches primarily categorize into *training-based* and *traversal-based* methods. Training-based models, such as GNNs/GCNs [2], [55], [56], [57], optimize neural networks to learn fraud patterns but often lack adaptivity to evolving tactics. Conversely, traversal-based methods (e.g., Label Propagation [11], PPR [12], [1]) use random walks to identify fraudulent structures directly, offering better adaptability. However, both categories typically assume centralized data, overlooking privacy and bandwidth constraints in multi-institutional settings. While recent collaborative attempts exist, such as SecureFD [58] (private GNNs) and CSGM [44] (scatter-gather mining), they either incur prohibitive latency overheads or lack rigorous privacy guarantees and generalizability to diverse fraud patterns.

Privacy-Preserving Graph Processing. Techniques for securing graph analytics generally rely on *cryptography* or *differential privacy (DP)*. Cryptographic methods, including MPC and Homomorphic Encryption [59], [60], [61], offer strong security but introduce massive computational and bandwidth costs, rendering them impractical for large-scale, low-latency detection. Alternatively, DP [62] offers a lightweight theoretical guarantee. While DP has been applied to specific algorithms (e.g., shortest path [63]) or general systems (e.g., PGPregel [13]), these approaches fail to jointly optimize high accuracy and low communication required for financial risk scoring. Other DP applications, such as graph publishing [53], [64] or model training [52], address orthogonal problems and do not support real-time collaborative graph processing.

VIII. Conclusion

CFDGraph is a privacy-preserving graph processing system specifically designed for collaborative fraud detection. By integrating DP with adaptive quantization, accuracy-aware noise mitigation, and communication compression, CFDGraph resolves the conflict between data privacy, detection accuracy, and network efficiency. Evaluations on real-world graphs demonstrate that CFDGraph achieves 100% recall for top-ranked risky accounts while reducing investigation costs by 99.5%. We acknowledge three limitations: (1) **Scope of detection:** CFDGraph relies on the propagation of risk scores across institutions. Therefore, patterns confined to a single institution may evade detection. (2) **Threat model:** The system operates under an honest-but-curious assumption. Robustness against malicious participants requires Byzantine-fault-tolerant extensions. (3) **Scalability:** While efficient for typical financial consortia (5-20 institutions), the privacy utility degrades as the number of participants grows large due to budget splitting, necessitating future work on hierarchical aggregation.

Acknowledgements

This work is supported by Guangdong and Hong Kong Universities "1+1+1" Joint Research Collaboration Scheme (No. 2025A0505000012) and a research fund from Ant Group; "ANR 22-PECY-0002" IPoP project (Interdisciplinary Project on Privacy) of the Cybersecurity PEPR; NSFC (62272315), Guangdong Province Quality Engineering Project (SZU Academic Affairs [2022] No. 7), Key Laboratory (2017B030314073) and Double-Hundred Action Project (892007987). Amelie Chi Zhou, Yuhong Feng and Lichun Li are the corresponding authors.

AI-Generated Content Acknowledgement

We employed GPT-5 to refine the language of the entire manuscript. All generative outputs were manually reviewed and edited for technical accuracy.

References

[1] F. Liu, Z. Li, B. Wang, J. Wu, J. Yang, J. Huang, Y. Zhang, W. Wang, S. Xue, S. Nepal, *et al.*, "eriskcom: an e-commerce risky community detection platform," *The VLDB Journal*, vol. 31, no. 5, pp. 1085–1101, 2022.

[2] G. Zhang, Z. Li, J. Huang, J. Wu, C. Zhou, J. Yang, and J. Gao, "efraudcom: An e-commerce fraud detection system via competitive graph neural networks," *ACM Transactions on Information Systems (TOIS)*, vol. 40, no. 3, pp. 1–29, 2022.

[3] H. Weng, S. Ji, F. Duan, Z. Li, J. Chen, Q. He, and T. Wang, "Cats: Cross-platform e-commerce fraud detection," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pp. 1874–1885, 2019.

[4] The People's Bank of China, "Payment System Report (2022)." http://www.pbc.gov.cn/en/3688247/3688978/3709143/4830432/2023032714461871866.pdf, 2024. Accessed onMarch 14, 2026.

[5] Z. Gyongyi, H. Garcia-Molina, and J. Pedersen, "Combating web spam with trustrank," in *30th International Conference on Very Large Data Bases (VLDB 2004)*, August 2004.

[6] U. Kang, H. Tong, J. Sun, C.-Y. Lin, and C. Faloutsos, "gbase: an efficient analysis platform for large graphs," *The VLDB Journal*, vol. 21, p. 637–650, Oct. 2012.

[7] C. Liu, L. Sun, X. Ao, J. Feng, Q. He, and H. Yang, "Intention-aware heterogeneous graph attention networks for fraud transactions detection," in *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pp. 3280–3288, 2021.

[8] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proceedings of the 29th ACM international conference on information & knowledge management*, pp. 315–324, 2020.

[9] J. Wang, R. Wen, C. Wu, Y. Huang, and J. Xiong, "Fdgars: Fraudster detection via graph convolutional networks in online app review system," in *Companion proceedings of the 2019 World Wide Web conference*, pp. 310–316, 2019.

[10] X. Sun, W. Feng, S. Liu, Y. Xie, S. Bhatia, B. Hooi, W. Wang, and X. Cheng, "Monlad: Money laundering agents detection in transaction streams," in *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*, WSDM '22, (New York, NY, USA), p. 976–986, Association for Computing Machinery, 2022.

[11] Z. Li, X. Chen, J. Song, and J. Gao, "Adaptive label propagation for group anomaly detection in large-scale networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12053–12067, 2023.

[12] T. H. Haveliwala, "Topic-sensitive pagerank," in *Proceedings of the 11th International Conference on World Wide Web*, WWW '02, (New York, NY, USA), p. 517–526, Association for Computing Machinery, 2002.

[13] A. C. Zhou, R. Qiu, T. Lambert, T. Allard, S. Ibrahim, and A. El Abbadi, "Pgpregel: an end-to-end system for privacy-preserving graph processing in geo-distributed data centers," in *Proceedings of the 13th Symposium on Cloud Computing*, SoCC '22, p. 386–402, 2022.

[14] F. Effendi and A. Chattopadhyay, "Privacy-preserving graph-based machine learning with fully homomorphic encryption for collaborative anti-money laundering," *arXiv preprint arXiv:2411.02926*, 2024.

[15] K. Evan, "Vpns for financial services," in *Insurance Technology Handbook*, pp. 18–1, CRC Press, 2018.

[16] A. Khazane, J. Rider, M. Serpe, A. Gogoglou, K. Hines, C. B. Bruss, and R. Serpe, "Deeptrax: Embedding graphs of financial transactions," in *2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA)*, pp. 126–133, 2019.

[17] A. C. Zhou, S. Ibrahim, and B. He, "On achieving efficient data transfer for graph processing in geo-distributed datacenters," in *2017 IEEE 37th international conference on distributed computing systems (ICDCS)*, pp. 1397–1407, IEEE, 2017.

[18] A. C. Zhou, B. Shen, Y. Xiao, S. Ibrahim, and B. He, "Cost-aware partitioning for efficient large graph processing in geo-distributed datacenters," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 7, pp. 1707–1723, 2019.

[19] A. C. Zhou, J. Luo, R. Qiu, H. Tan, B. He, and R. Mao, "Adaptive partitioning for large-scale graph analytics in geo-distributed data centers," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*, pp. 2818–2830, IEEE, 2022.

[20] H. Tan, Y. Xiao, A. C. Zhou, K. Lu, and X. Yang, "Distributed and adaptive partitioning for large graphs in geo-distributed data centers," *IEEE Transactions on Parallel and Distributed Systems*, 2025.

[21] F. Yao, Q. Tao, W. Yu, Y. Zhang, S. Gong, Q. Wang, G. Yu, and J. Zhou, "Ragraph: A region-aware framework for geo-distributed graph processing," *Proceedings of the VLDB Endowment*, vol. 17, no. 3, pp. 264–277, 2023.

[22] G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, "Pregel: a system for large-scale graph processing," in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pp. 135–146, 2010.

[23] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: a survey," *Data mining and knowledge discovery*, vol. 29, pp. 626–688, 2015.

[24] J. M. Kleinberg *et al.*, "Authoritative sources in a hyperlinked environment.," in *SODA*, vol. 98, pp. 668–677, Citeseer, 1998.

[25] G. Oded, *Foundations of Cryptography: Volume 2, Basic Applications*. USA: Cambridge University Press, 1st ed., 2009.

[26] J. Brickell and V. Shmatikov, "Privacy-preserving graph algorithms in the semi-honest model," in *International Conference on the Theory and Application of Cryptology and Information Security*, 2005.

[27] J. Yu, K. Chen, Y. Chen, X. Fan, X. Zhu, C. Hong, and W. Chen, "Graphace: Secure two-party graph analysis achieving communication efficiency," in *USENIX Security Symposium*, 2025.

[28] M. Dong, C. Zhang, Y. Bai, and Y. Chen, "Efficient multi-party private set union without non-collusion assumptions," in *USENIX Security Symposium*, 2025.

[29] H. Wu, C. Wei, Y. Wang, L. Lin, Y. Leng, S. He, M. Zhao, H. Wu, Y. Yan, and A. Zhou, "Zero-knowledge verifiable graph query evaluation via expansion-centric operator decomposition," *ArXiv*, vol. abs/2507.00427, 2025.

[30] M. Hay, C. Li, G. Miklau, and D. Jensen, "Accurate estimation of the degree distribution of private networks," in *2009 Ninth IEEE International Conference on Data Mining*, pp. 169–178, IEEE, 2009.

[31] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," *2017 IEEE Symposium on Security and Privacy (SP)*, pp. 3–18, 2017.

[32] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of cryptography conference*, pp. 265–284, Springer, 2006.

[33] K. Imakubo, Y. Soejima, *et al.*, "The transaction network in japan's interbank money markets," *Monetary and Economic Studies*, vol. 28, pp. 107–150, 2010.

[34] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii, "Scalable k-means++," *Proc. VLDB Endow.*, vol. 5, p. 622–633, mar 2012.

[35] D. Su, J. Cao, N. Li, E. Bertino, M. Lyu, and H. Jin, "Differentially private k-means clustering and a hybrid approach to private optimization," *ACM Trans. Priv. Secur.*, vol. 20, oct 2017.

[36] Y. Li, X. Dong, and W. Wang, "Additive powers-of-two quantization: An efficient non-uniform discretization for neural networks," 2020.

[37] J. Yang and J. Leskovec, "Defining and evaluating network communities based on ground-truth," in *Proceedings of the ACM SIGKDD workshop on mining data semantics*, pp. 1–8, 2012.

[38] Z. Chen, F. Zhang, J. Guan, J. Zhai, X. Shen, H. Zhang, W. Shu, and X. Du, "CompressGraph: Efficient parallel graph analytics with rule-based compression," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, pp. 1–31, 2023.

[39] Z. Chen, F. Zhang, Y. Xia, W. Zhang, X. Zhu, W. Chen, and X. Du, "CompressGNN: Accelerating graph neural network training via hierarchical compression," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 286–297, 2025.

[40] L. Feng, F. Zhang, Z. Chen, Y. Tang, J. Guan, X. Zhu, and X. Du, "Enabling efficient update on rule-based compressed graph," *SIGMOD*, 2026.

[41] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, p. 211–407, aug 2014.

[42] L. Page, "The pagerank citation ranking: Bringing order to the web," tech. rep., Technical Report, 1999.

[43] A. T. Mughrabi, M. Ibrahim, and G. T. Byrd, "Qpr: Quantizing pagerank with coherent shared memory accelerators," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 962–972, IEEE, 2021.

[44] Z. Tian, Y. Ding, W. Qu, X. Yu, E. Gong, J. Zhang, J. Liu, and K. Ren, "Towards collaborative anti-money laundering among financial institutions," in *THE WEB CONFERENCE 2025 (WWW'25)*, 2025.

[45] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection." http://snap.stanford.edu/data, June 2014.

[46] Y. Che, "Kronecker graph generator (forked from graph500, supporting a binary edge list format)." https://github.com/RapidsAtHKUST/Graph500KroneckerGraphGenerator, 2020.

[47] P. Boldi and S. Vigna, "The webgraph framework i: compression techniques," in *Proceedings of the 13th International Conference on World Wide Web*, WWW '04, (New York, NY, USA), p. 595–602, Association for Computing Machinery, 2004.

[48] S. Kumar, F. Spezzano, V. Subrahmanian, and C. Faloutsos, "Edge weight prediction in weighted signed networks," in *Data Mining (ICDM), 2016 IEEE 16th International Conference on*, pp. 221–230, IEEE, 2016.

[49] T. Suzumura and H. Kanezashi, "Anti-Money Laundering Datasets: InPlusLab anti-money laundering datadatasets." http://github.com/IBM/AMLSim/, 2021.

[50] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, (Red Hook, NY, USA), Curran Associates Inc., 2023.

[51] R. Lempel and S. Moran, "Salsa: the stochastic approach for link-structure analysis," *ACM Transactions on Information Systems (TOIS)*, vol. 19, no. 2, pp. 131–160, 2001.

[52] T. Tang, Y. Niu, S. Avestimehr, and M. Annavaram, "Edge private graph neural networks with singular value perturbation," *Proc. Priv. Enhancing Technol.*, vol. 2024, no. 3, pp. 391–406, 2024.

[53] Q. Yuan, Z. Zhang, L. Du, M. Chen, P. Cheng, and M. Sun, "Privgraph: differentially private graph data publication by exploiting community information," in *Proceedings of the 32nd USENIX Conference on Security Symposium*, SEC '23, (USA), USENIX Association, 2023.

[54] A. Malyshev, "High-volume transactions: Lessons from the largest payment systems." https://sdk.finance/blog/high-volume-transactions-lessons-from-the-largest-payment-systems/#pll_switcher, 2025.

[55] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proceedings of the 2019 SIAM international conference on data mining*, pp. 594–602, SIAM, 2019.

[56] Z. Peng, M. Luo, J. Li, L. Xue, and Q. Zheng, "A deep multi-view framework for anomaly detection on attributed networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 6, pp. 2539–2552, 2020.

[57] F. Xiao, Y. Wu, M. Zhang, G. Chen, and B. C. Ooi, "Mint: Detecting fraudulent behaviors from time-series relational data," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3610–3623, 2023.

[58] X. Liu, X. Fan, R. Ma, K. Chen, Y. Li, G. Wang, and W. Xu, "Collaborative fraud detection on large scale graph using secure multi-party computation," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pp. 1473–1482, 2024.

[59] M. Du, S. Wu, Q. Wang, D. Chen, P. Jiang, and A. Mohaisen, "Graphshield: Dynamic large graphs for secure queries with forward privacy," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 7, pp. 3295–3308, 2020.

[60] E. Ghosh, S. Kamara, and R. Tamassia, "Efficient graph encryption scheme for shortest path queries," in *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, pp. 516–525, 2021.

[61] S. Wang, Y. Zheng, X. Jia, and X. Yi, "Pegraph: A system for privacy-preserving and efficient search over encrypted social graphs," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3179–3194, 2022.

[62] C. Dwork, A. Roth, *et al.*, "The algorithmic foundations of differential privacy," *Foundations and Trends® in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[63] A. Sealfon, "Shortest paths and distances with differential privacy," in *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, pp. 29–41, 2016.

[64] S. Zhang, H. Hu, Q. Ye, and J. Xu, "Privdpr: Synthetic graph publishing with deep pagerank under differential privacy," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, KDD '25, (New York, NY, USA), p. 1936–1947, Association for Computing Machinery, 2025.